Surtr: Transparent Verification with Simple yet Strong Coercion Mitigation

Rosario Giustolisi [1], Maryam Sheikhi Garjan [2] und Peter Browne Rønne [3]

Abstract: Transparent verification allows voters to directly identify their vote in cleartext in the final tally result. Both Selene and Hyperion offer this simple and intuitive verification method, and at the same time allow for coercion to be mitigated under the assumption that tally servers can privately notify voters of the keying material needed for verification. Subsequently, a voter can generate fake keying material to deceive a coercer.

In this paper, we propose Surtr, a new scheme that enables transparent verification without requiring a private notification channel. This approach strengthens coercion mitigation, since a coercer can monitor the notification channel, and simplifies the process by eliminating the need for voters to generate fake keying material for the coercer.

Keywords: End-to-end verifiability, Vote privacy, Coercion resistance

1 Introduction

Transparent verification is a desirable property of voting schemes, enabling voters to independently verify that their vote has been correctly included in the final tally. Specifically, it allows voters to identify their vote in cleartext within the published election result, without having to perform checks on encrypted ballots. This form of verification is simple and intuitive, enabling voters to build trust in the electoral process without requiring specialised tools or third-party assistance.

Voting systems such as Selene [RRI16], the CNRS scheme [ACW13], sElect [Kü16] and Hyperion [Da24] incorporate transparent verification. In particular, Selene and Hyperion offer mitigation mechanisms to coercion threats that arise from the transparent verification process. In these mechanisms, voters are privately notified by the tellers of the keying material required to verify their votes. A coerced voter can then generate fake keying material yielding a plausible decrypted vote, deceiving the coercer. However, these mechanisms assume that both the notification channel and the generation of fake key material remain hidden from the coercer.

In this paper, we introduce Surtr, a new scheme that offers coercion mitigation and transparent verification without relying on a private notification channel. By removing this channel,

[1] IT University of Copenhagen, Denmark, rosg@itu.dk, https://orcid.org/0000-0000-0000-0000
[2] University of Hamburg, Germany, maryam.sheikhi.garjan@uni-hamburg.de, https://orcid.org/0000-0000-0000-0000
[3] University of Luxembourg, Luxembourg, peter.roenne@uni.lu, https://orcid.org/0000-0000-0000-0000

Surtr eliminates the risk posed by a coercer monitoring notifications. Moreover, it simplifies the overall coercion-mitigation strategy by relieving voters of the burden to fabricate fake keying material.

Contributions. The main contribution of this paper is the design of a voting scheme that achieves both coercion mitigation and transparent verification without relying on a private notification channel. This allows for a simpler and stronger coercion model, where the coercer may fully control the voter's device after the vote has been cast. Surtr is based on standard mix-net constructions, and we formally prove that it satisfies ballot privacy under a modified game-based definition derived from Hyperion, adapted to exclude the assumption of a private notification channel. We also prove that Surtr ensures verifiability in presence of a malicious bulletin board and dishonest tallying servers, assuming that either the casting device or the verification device is trusted. We provide a prototype implementation of Surtr in Python and demonstrate that it is efficient for use in large-scale elections.

2 Related Work

Several voting schemes provide strong coercion guarantees [AFT08; CGY22; CH12; GGS24; JCJ10; LHK16; Sp12] at the cost of transparent verification. A few scheme have offered transparent verification to allow voters to directly track and verify the plaintext vote in the final tally result, starting from a boardroom voting scheme [ACW13] where the voters themselves create trackers. Also sElect [Kü16] used partially voter-generated trackers to achieve accountability. However, these schemes do not provide coercion mitigation, since the voter could announce the tracker to a coercer even before the tally was created. Selene [RRI16] improves on this and achieves coercion-mitigation, by first notifying the voters about the trackers after the tally is published, allowing the voter to equivocate to another tracker for a coercer's desired vote. However, the cryptographic terms in the notification need to be sent via a private channel to allow equivocation of the tracker in the presence of a coercer. Hyperion [Da24] removes the need for trackers and replaces them with Diffie-Hellman terms that the voter can compute to identify their plaintext vote but again using a private channel for notification. The advantage of this construction was highlighted by [Mo24] which achieved everlasting ballot privacy and everlasting receipt-freeness. The usefulness of these schemes was demonstrated both in user studies [AS20; Di19; Zo21] and real world trials [Sa20].

Surtr has two main differences to these schemes: 1) Surtr does not require a private channel for the notification of the dual key; and 2) Surtr does not require the voter to generate a fake dual key at notification. In fact, after voting the coercer can fully control the voter's device.

A third advantage compared to Selene is that a tracker collision cannot occur: in Selene the voter might equivocate a tracker to the coercer's tracking number. This is countered here by having an extra set of votes, one for each candidate per voter. The idea of such extra

ballots was also proposed in a variant of Selene [RRI16]. However, the use in Surtr is quite different, and the extra ballots constitute an essential ingredient in the construction of Surtr where the ordering of ballots will determine the real cast vote.

3 Surtr

$$ID_1, PK_1, \{\mathbf{v_1}\}, \{v_2\}, \{g^{x_{11}}\}, \{g^{x_{12}}\}, \sigma_1, \pi_1$$
$$ID_2, PK_2, \{\mathbf{v_2}\}, \{v_1\}, \{g^{x_{21}}\}, \{g^{x_{22}}\}, \sigma_2, \pi_2$$
$$ID_3, PK_3, \{\mathbf{v_2}\}, \{v_1\}, \{g^{x_{31}}\}, \{g^{x_{32}}\}, \sigma_3, \pi_3$$
$$ID_4, PK_4, \{\mathbf{v_1}\}, \{v_2\}, \{g^{x_{41}}\}, \{g^{x_{42}}\}, \sigma_4, \pi_4$$
$$ID_5, PK_5, \{\mathbf{v_1}\}, \{v_2\}, \{g^{x_{51}}\}, \{g^{x_{52}}\}, \sigma_5, \pi_5$$

Abb. 1: Each voter casts an encrypted vote per each candidate. The first encrypted vote is the voter's choice (shown in **bold**). Each encrypted vote is paired with an encrypted commitment. In this example, there are two candidates and five voters. Voter ID_1 votes for candidate v_1 whose associated commitment is $g^{x_{11}}$. All encryptions are performed under the public key of the tellers and, together with the NIZKP proofs, form the voter's ballot, which is signed by the voter.

3.1 Overview

Our technique considers a *voter*, who casts ballots, and the *tally servers* in charge of tallying and publishing the final election result on the bulletin board. The tally servers form a t-out-of-n threshold encryption system. Each voter's ballot contains the encryptions of *all* candidates, with the voter's actual choice being the first of these encryptions. The ballot also contains encryptions of commitments to ephemeral *trapdoor keys*, each associated with an encryption of a candidate (i.e. the first encrypted commitment is associated to the first encrypted vote, which is the voter's choice). Figure 1 depicts an example of the voting phase.

A key idea introduced already here is that the ballot contains encryptions of all candidates. After the voter chooses a candidate, the voter's device sets the first encrypted vote as the voter's choice and displays the trapdoor keys next to corresponding candidates. These trapdoor keys can later be used by the voter to verify their vote. Note that the associations between candidates and trapdoor keys can be shared with a coercer, as they do not reveal the voter's choice.

After the voting phase, the tally servers raise all encrypted commitments on a voter's ballot to the same random exponent and re-encrypt them. The randomized and re-encrypted commitments, along with an encryption of the *dual key* (a commitment to the random exponent), are appended next to each voter's ballot. The tally servers also append a triplet of encryptions consisting of a re-encryption of the voter's choice (i.e. the first encrypted vote), a new random commitment, and a new dual key. This extension of the voter ballots is shown in Figure 2.

$ID_1, PK_1, \{\mathbf{v_1}\}, \{v_2\}, \{g^{x_{11}}\}, \{g^{x_{12}}\}, \sigma_1, \pi_1, \{g^{x_{11}r_1}\}', \{g^{x_{12}r_1}\}', \{g^{r_1}\}, (\{v_1\}', \{g^{y_1}\}, \{g^{s_1}\}), \pi_{T_1}$

$ID_2, PK_2, \{\mathbf{v_2}\}, \{v_1\}, \{g^{x_{21}}\}, \{g^{x_{22}}\}, \sigma_2, \pi_2, \{g^{x_{21}r_2}\}', \{g^{x_{22}r_2}\}', \{g^{r_2}\}, (\{v_2\}', \{g^{y_2}\}, \{g^{s_2}\}), \pi_{T_2}$

$ID_3, PK_3, \{\mathbf{v_2}\}, \{v_1\}, \{g^{x_{31}}\}, \{g^{x_{32}}\}, \sigma_3, \pi_3, \{g^{x_{31}r_3}\}', \{g^{x_{32}r_3}\}', \{g^{r_3}\}, (\{v_2\}', \{g^{y_3}\}, \{g^{s_3}\}), \pi_{T_3}$

$ID_4, PK_4, \{\mathbf{v_1}\}, \{v_2\}, \{g^{x_{41}}\}, \{g^{x_{42}}\}, \sigma_4, \pi_4, \{g^{x_{41}r_4}\}', \{g^{x_{42}r_4}\}', \{g^{r_4}\}, (\{v_1\}', \{g^{y_4}\}, \{g^{s_4}\}), \pi_{T_4}$

$ID_5, PK_5, \{\mathbf{v_1}\}, \{v_2\}, \{g^{x_{51}}\}, \{g^{x_{52}}\}, \sigma_5, \pi_5, \{g^{x_{51}r_5}\}', \{g^{x_{52}r_5}\}', \{g^{r_5}\}, (\{v_1\}', \{g^{y_5}\}, \{g^{s_5}\}), \pi_{T_5}$

Abb. 2: The tally servers append several encryptions to each voter's ballot: the randomized and re-encrypted commitments, one for each per candidate, along with an encrypted dual key, and a triplet consisting of a re-encryption of the voter's choice, a new random commitment, and another fresh dual key.

(a)	(b)	(c)
$\{\mathbf{v_1}\}, \{g^{x_{11}r_1}\}', \{g^{r_1}\}$	$\{v_1\}'', \{g^{y_4}\}', \{g^{s_4}\}'$	v_2, g^{y_3}, g^{s_3}
$\{v_2\}, \{g^{x_{12}r_1}\}', \{g^{r_1}\}$	$\{\mathbf{v_2}\}', \{g^{x_{31}r_3}\}'', \{g^{r_3}\}'$	$\mathbf{v_1}, g^{x_{41}r_4}, g^{r_4}$
$\{\mathbf{v_2}\}, \{g^{x_{21}r_2}\}', \{g^{r_2}\}$	$\{v_2\}', \{g^{x_{42}r_4}\}'', \{g^{r_4}\}'$	$v_2, g^{x_{42}r_4}, g^{r_4}$
$\{v_1\}, \{g^{x_{22}r_2}\}', \{g^{r_2}\}$	$\{v_1\}'', \{g^{y_5}\}', \{g^{s_5}\}'$	$\mathbf{v_2}, g^{x_{21}r_2}, g^{r_2}$
$\{\mathbf{v_2}\}, \{g^{x_{31}r_3}\}', \{g^{r_3}\}$	$\{v_1\}'', \{g^{y_1}\}', \{g^{s_1}\}'$	v_1, g^{y_1}, g^{s_1}
$\{v_1\}, \{g^{x_{32}r_3}\}', \{g^{r_3}\}$	$\{\mathbf{v_1}\}', \{g^{x_{41}r_4}\}'', \{g^{r_4}\}'$	$\mathbf{v_2}, g^{x_{31}r_3}, g^{r_3}$
$\{\mathbf{v_1}\}, \{g^{x_{41}r_4}\}', \{g^{r_4}\}$	$\{\mathbf{v_1}\}', \{g^{x_{11}r_1}\}'', \{g^{r_1}\}'$	v_2, g^{y_2}, g^{s_2}
$\{v_2\}, \{g^{x_{42}r_4}\}', \{g^{r_4}\}$	$\{v_1\}', \{g^{x_{22}r_2}\}'', \{g^{r_2}\}'$	$\mathbf{v_1}, g^{x_{51}r_5}, g^{r_5}$
$\{\mathbf{v_1}\}, \{g^{x_{51}r_5}\}', \{g^{r_5}\}$	$\{v_2\}', \{g^{x_{12}r_1}\}'', \{g^{r_1}\}'$	v_1, g^{y_5}, g^{s_5}
$\{v_2\}, \{g^{x_{52}r_5}\}', \{g^{r_5}\}$	$\{v_2\}'', \{g^{y_2}\}', \{g^{s_2}\}'$	$v_2, g^{x_{52}r_5}, g^{r_5}$
$\{v_1\}', \{g^{y_1}\}, \{g^{s_1}\}$	$\{\mathbf{v_1}\}', \{g^{x_{51}r_5}\}'', \{g^{r_5}\}'$	$v_1, g^{x_{32}r_3}, g^{r_3}$
$\{v_2\}', \{g^{y_2}\}, \{g^{s_2}\}$	$\{v_2\}'', \{g^{y_3}\}', \{g^{s_3}\}'$	$v_1, g^{x_{22}r_2}, g^{r_2}$
$\{v_2\}', \{g^{y_3}\}, \{g^{s_3}\}$	$\{\mathbf{v_2}\}', \{g^{x_{21}r_2}\}'', \{g^{r_2}\}'$	$\mathbf{v_1}, g^{x_{11}r_1}, g^{r_1}$
$\{v_1\}', \{g^{y_4}\}, \{g^{s_4}\}$	$\{v_1\}', \{g^{x_{32}r_3}\}'', \{g^{r_3}\}'$	$v_2, g^{x_{12}r_1}, g^{r_1}$
$\{v_1\}', \{g^{y_5}\}, \{g^{s_5}\}$	$\{v_2\}', \{g^{x_{52}r_5}\}'', \{g^{r_5}\}'$	v_1, g^{y_4}, g^{s_4}

Abb. 3: Each triplet (step (a)) consisting of the encrypted vote, re-encrypted commitment, and encrypted dual key is subjected to a re-encryption mixnet (step (b)). Then, the tally servers run a decryption mixnet to shuffle and decrypt votes, commitments, and dual keys (step (c)). Voter ID_1 uses the trapdoor x_{11} to verify the vote by finding the triplet in step (c), which contains the vote, and confirming that the third element, when raised to x_{11}, gives the second element. This verifies the first element, which is the plaintext vote. The voter can deceive the coercer by using the trapdoor x_{12}. Note that, for each candidate, the tally includes a number of extra votes equal to the number of voters. Therefore this amount should be subtracted to obtain the final tally.

The tally servers form the triplets containing an encrypted vote with the corresponding re-encrypted commitment and encrypted dual key, as depicted in Figure 3(a). These triplets are then subjected to a re-encryption mixnet [TW10], resulting in Figure 3(b). Finally, all triplets undergo a decryption mixnet that shuffles and decrypts votes, commitments, and dual keys, as in Figure 3(c).

The voter can verify their vote by raising each dual key to the trapdoor key associated with their choice. Similarly, the voter can mislead a coercer by raising the dual key to any other trapdoor key. To avoid requiring the voter to find the dual key related to their ballot, the tallying servers can notify the dual key directly to the voter. Even if a coercer learns the dual key, it poses no problem since the voter can use a trapdoor key associated with any other candidate to deceive the coercer.

The final tally is computed by removing v votes from each candidate's total, where v is the number of voters. An alternative that would not require this removal, but would allow for immediate tallying, is for the tally servers to extend all voter ballots with the same fresh dual key. In this case, all and only the re-encrypted voter choices would be associated with the same dual key, enabling immediate tallying.

The main idea behind our technique is that the ballot contains encryptions of all candidates plus a re-encryption of the voter's choice. Correctness is guaranteed because the re-encryption contains the voter's choice. Individual verifiability is ensured by the fact that the first encrypted vote is the voters' choice. Since the voter's commitments are first raised to the same random exponent and then subjected to re-encryption and decryption mixes, the voter can use the trapdoor key associated to their choice to directly verify that their vote is included in the tally and use any of the other trapdoor keys to deceive a coercer. This neither requires any private communication from the tally servers to the voter nor requires the voter to generate fake dual keys. Moreover, our technique removes the need for individual views via exponentiation mixnets as in Hyperion.

3.2 Cryptographic primitives

The cryptographic primitives required in Surtr include a digital signature scheme, threshold encryption, verifiable re-encryption and decryption mix-nets, and non-interactive zero-knowledge proofs of knowledge (NIZKPoK). The encryption scheme is defined by the standard algorithms $(\mathsf{EKeyGen}, \mathsf{Enc}, \mathsf{Dec})$, which specify key generation, encryption, and decryption, respectively. The digital signature scheme is defined by the standard triplet $(\mathsf{SKeyGen}, \mathsf{Sign}, \mathsf{SVerify})$, used to authenticate ballots of the voters. The algorithm $(\mathbf{x}, \mathbf{g}^{\mathbf{x}}) \xleftarrow{\$} \mathsf{TKeyGen}(pp, pk)$ generates sets of trapdoor keys $\mathbf{x} = \{x_0, \ldots x_{N-1}\}$ and commitments $\mathbf{g}^{\mathbf{x}} = \{g^{x_0}, \ldots, g^{x_{N-1}}\}$ given a public representation under public parameters pp and public key pk. We assume a (t, n)-threshold encryption scheme without a trusted authority, supporting verifiable decryption. Any subset of t out of n decryption parties

can collaboratively decrypt a ciphertext and produce a publicly verifiable proof of correct decryption.

In addition, we use re-encryption and decryption mix-nets to shuffle ciphertexts while preserving privacy. These are accompanied by publicly verifiable proofs of correctness. Specifically, we rely on parallel verifiable shuffle protocols to ensure the integrity of mixnet operations.

To guarantee verifiability, we employ a system of NIZK proofs of knowledge $(\mathsf{Proof}_R, \mathsf{Verify}_R)$ for various relations, including: proof of plaintext knowledge, proof of secret key knowledge, proof of correct re-encryption and shuffling, proof of correct decryption. These proofs ensure that all steps in the protocol are executed correctly without revealing private information.

3.3 Formal Description

The algorithms that define Surtr are as follows.

- $\mathsf{EASetup}(1^\lambda, \mathbb{I}, \mathbb{V}) \rightarrow (pk_{EA}, sk_{EA})$: on input of the security parameter 1^λ, electoral roll $\mathbb{I}$, and candidate list $\mathbb{V}$ generates public parameters pp, and computes $(pk_{EA}, sk_{EA}) \xleftarrow{\$} \mathsf{EKeyGen}(pp)$.
- $\mathsf{Setup}(id, pk_{EA}) \rightarrow (id, (\mathbf{x}, \mathbf{g^x}, sk), (\mathbf{ctr}, \pi_{\mathbf{ctr}}, pk))$: on input of the voter identity id and pk_{EA}, which contains pp, computes $(sk, pk) \xleftarrow{\$} \mathsf{SKeyGen}(pp)$, where pk contains a proof of knowledge π_{sk}, and run $(\mathbf{x}, \mathbf{g^x}) \xleftarrow{\$} \mathsf{TKeyGen}(pp, pk)$. Then, it computes $ctr_i \xleftarrow{\$} \mathsf{Enc}(pk_{EA}, g^{x_i}; r_{x_i})$ for $i \in [0, N-1]$, and sets $\mathbf{ctr} = (ctr_0, ctr_1, \ldots, ctr_{N-1})$. It runs $\pi_{\mathbf{ctr}} \xleftarrow{\$} \mathsf{Proof}_{ctr}(\chi, \omega)$, where χ=($\mathbf{ctr}$, pk_{EA}), ω=($\mathbf{x}$, $\mathbf{r_x}$) s.t. $(\chi, \omega) \in R^{\text{ctr}}$ iff

$$ctr_0 = \mathsf{Enc}(pk_{EA}, g^{x_0}; r_{x_0}) \wedge \ldots \wedge ctr_{N-1} = \mathsf{Enc}(pk_{EA}, g^{x_{N-1}}; r_{x_{N-1}}).$$

- $\mathsf{Vote}(id, sk, pk_{EA}, (v_0, \mathbf{ctr}, \pi_{\mathbf{ctr}})) \rightarrow \beta$: on input voter identity $id \in \mathbb{I}$, voter secret sk, tallying servers public key pk_{EA}, vote option $v_0 \in \mathbb{V}$, encrypted commitments $\mathbf{ctr}$, proofs $\pi_{\mathbf{ctr}}$, and implicit input $(\mathbb{V}, \mathbb{I})$ where $N = \mid \mathbb{V} \mid$
  1. Compute $ct_0 \xleftarrow{\$} \mathsf{Enc}(pk_{EA}, v_0; r_{v_0})$
  2. Set $\mathbb{W} = \mathbb{V} \setminus \{v_0\}$
  3. Compute **for** $i \in [1, N-1]$ **do**
     $v \xleftarrow{\$} \mathbb{W}$
     $ct_i \xleftarrow{\$} \mathsf{Enc}(pk_{EA}, v; r_{v_i})$
     $\mathbb{W} = \mathbb{W} \setminus \{v\}$
  4. Set $\mathbf{ct} = (ct_0, ct_1, \ldots, ct_{N-1})$, $\mathbf{v} = (v_0, v_1, \ldots, v_{N-1})$ and encryption randomness $\mathbf{r_v} = (r_{v_0}, r_{v_1}, \ldots, r_{v_{N-1}})$.

5. Run $\pi_v \xleftarrow{\$} \mathsf{Proof}_{vote}(\chi, \omega)$, where χ=($\mathbf{ct}, pk_{EA}, \mathbb{V}$), ω=($\mathbf{v}, \mathbf{r_v}$) s.t. $(\chi, \omega) \in R^{\text{vote}}$ iff

$$ct_0 = \mathsf{Enc}(pk_{EA}, v_0; r_{v_0}) \wedge \ldots \wedge ct_{N-1} = \mathsf{Enc}(pk_{EA}, v_{N-1}; r_{v_{N-1}}) \wedge$$
$$v_0 \in \mathbb{V} \wedge \ldots \wedge v_{N-1} \in \mathbb{V} \wedge \mathbf{v} \equiv \mathbb{V}$$

6. Set $\pi = (\pi_v, \pi_{\mathbf{ctr}})$ and compute $\sigma \xleftarrow{\$} \mathsf{Sign}(sk, (\mathbf{ct}, \mathbf{ctr}, \pi))$ and return $\beta = (id, pk, \mathbf{ct}, \mathbf{ctr}, \pi, \sigma)$.

- $\mathsf{Validate}(\mathcal{BB}, \beta) \rightarrow \top/\bot$: on input a ballot $\beta = (id, pk, \mathbf{ct}, \mathbf{ctr}, \pi, \sigma)$ and implicit input $(pk_{EA}, \mathbb{I}, \mathbb{V})$ checks that i) $id \in \mathbb{I}$, ii) no terms in β already appear in $\mathcal{BB}$, iii) $\top \leftarrow \mathsf{SVerify}((id, pk, \mathbf{ct}, \mathbf{ctr}, \pi), \sigma))$, σ is a valid signature, and iv) $\top \leftarrow \mathsf{Verify}(\chi, \pi)$, where $\chi = (pk_{EA}, \mathbf{ct}, \mathbf{ctr})$. If any of the checks fail, it returns $\bot$ otherwise $\top$.
- $\mathsf{Append}(\mathcal{BB}, \beta) \rightarrow \mathcal{BB}$: on input a ballot $\beta = (id, pk, \mathbf{ct}, \mathbf{ctr}, \pi, \sigma)$ checks that $\mathsf{Validate}(\mathcal{BB}, \beta) = \top$. If so, it updates $\mathcal{BB}$ with β.
- $\mathsf{VerifyVote}(\mathcal{BB}, (id, pk), v, x_v) \rightarrow \bot/\top$: on input $\mathcal{BB}$, voter's $(id, pk)$and, optionally, voter option v and trapdoor key x_v
  1. Check that there is a ballot β with id on $\mathcal{BB}$ and that $\mathsf{Validate}(\mathcal{BB}, \beta) = \top$.
  2. If v and x_v are given, compute g^{tx_v} to recover the triplet $(v_i, g^{x_v t}, g^t)$ from $\mathcal{BB}$, and check that $v_i = v$.

  If any of the checks fail, it returns $\bot$ otherwise $\top$.
- $\mathsf{ExtendMix}(\mathcal{BB}, pk_{EA}, id) \rightarrow \beta_e$: on input $\mathcal{BB}$, pk_{EA}, and id, obtains the ballot $\beta = (id, pk, \mathbf{ct}, \mathbf{ctr}, \pi, \sigma)$ and then does the following:
  1. Compute $ctp \xleftarrow{\$} \mathsf{Enc}(pk_{EA}, g^t; r_t)$ where t is random.
  2. Compute **for** $i \in [0, N-1]$ **do** $ctt_i \leftarrow \mathsf{ReEnc}(pk_{EA}, ctr_i^t; r_i)$
  3. Compute $\widehat{ct} \xleftarrow{\$} \mathsf{ReEnc}(pk_{EA}, ct_0; r_e)$, $\widehat{ctr} \xleftarrow{\$} \mathsf{Enc}(pk_{EA}, g^y; r_y)$, and $\widehat{ctp} \xleftarrow{\$} \mathsf{Enc}(pk_{EA}, g^{t'}; r_{t'})$ where both y and t' are random.
  4. Run $\pi_T \xleftarrow{\$} \mathsf{Proof}_{ext}(\chi, \omega)$, where χ=($\beta, \mathbf{ctt}, pk_{EA}, \mathbb{V}, ctp, \widehat{ct}, \widehat{ctr}, \widehat{ctp}$), ω=($t, y, r_t, r_e, r_y, r_{t'}, \mathbf{r}$), $\mathbf{ctt}$=($ctt_0, ctt_1, \ldots, ctt_{N-1}$) and $\mathbf{r}$=($r_0, r_1, \ldots, r_{N-1}$) s.t. $(x, \omega) \in R^{\text{ext}}$ iff

$$ctt_0 = \mathsf{ReEnc}(pk_{EA}, ctr_0^t; r_0) \wedge \ldots \wedge$$
$$ctt_{N-1} = \mathsf{ReEnc}(pk_{EA}, ctr_{N-1}^t; r_{N-1}) \wedge$$
$$ctp = \mathsf{Enc}(pk_{EA}, g^t; r_t) \wedge \widehat{ct} = \mathsf{ReEnc}(pk_{EA}, ct_0; r_e) \wedge$$
$$\widehat{ctr} = \mathsf{Enc}(pk_{EA}, g^y; r_y) \wedge \widehat{ctp} = \mathsf{Enc}(pk_{EA}, g^{t'}; r_{t'})$$

  5. Set the extended ballot $\beta_e = (\beta, \mathbf{ctt}, \widehat{ct}, \widehat{ctr}, \widehat{ctp}, ctp, \pi_T)$ and run $\mathsf{Append}(\mathcal{BB}, \beta_e)$.
- $\mathsf{Tally}(\mathcal{BB}, sk_{EA}) \rightarrow (R, \Pi)$: on input $\mathcal{BB}$ and the decryption key sk_{EA} computes the election result as follows:

Let $\beta_e = (id, pk, \mathbf{ct}, \mathbf{ctr}, \pi, \sigma, \mathbf{ctt}, \widehat{ct}, \widehat{ctr}, \widehat{ctp}, ctp, \pi_T)$ be the extended ballot of the voter id in $\mathcal{BB}$. Let $(ct_i, ctt_i, ctp_i)_{i\in[0,N]} \in \beta_e$ where $(ct_N, ctt_N, ctp_N) = (\widehat{ct}, \widehat{ctr}, \widehat{ctp})$. Let n be the number of voters; there are $n(N+1)$ tuples $(ct_i, ctt_i, ctp_i) \in \mathcal{BB}$ for all voters that are denoted by $(\overline{\mathbf{ct}}, \overline{\mathbf{ctt}}, \overline{\mathbf{ctp}})$.

1. On input $(\overline{\mathbf{ct}}, \overline{\mathbf{ctt}}, \overline{\mathbf{ctp}})$ run a re-encryption mixnet to shuffle in parallel the ciphetexts and produce $\{(ct'_{i_j}, ctt'_{i_j}, ctp'_{i_j})\}_{j\in n(N+1)}$ and π_{r-enc}.
2. Run a decryption mixnet that generates a proof π_d and a set of $n(N+1)$ tuples of the form (v, com, dk) that contains a candidate v_i, a commitment $com_i = g^{x_i t_i}$, and dual key $dk_i = g^{t_i}$.
3. Sets $\Pi = (\{\pi_{T_i}\}_{i\in I}, \pi_{r-enc}, \pi_d)$. Let $(R_{v_0}, R_{v_1}, \ldots R_{v_{N-1}})$ denote the total number of votes for each candidate, then by subtracting n votes per candidate, the result is $R = (R_{v_0} - n, R_{v_1} - n, \ldots R_{v_{N-1}} - n)$.

- $\mathsf{VerifyTally}(\mathcal{BB}, (R, \Pi)) \rightarrow \perp/\top$: on input $\mathcal{BB}$, the tally result and corresponding proofs (R, Π), verifies the correctness of (R, Π) on $\mathcal{BB}$. If any of the checks fails, it returns $\perp$ otherwise $\top$.

The scheme is organised in the following four phases.

Setup phase: The algorithm $\mathsf{EASetup}(1^\lambda, \mathbb{I}, \mathbb{V})$ allows the tallying servers to generate their key pair (pk_{EA}, sk_{EA}). The algorithm $\mathsf{Setup}(id, pk_{EA})$ allows the voter to register to the election with the signing key pair (sk, pk). The voter also generates the sets of trapdoor keys $\mathbf{x}$, commitments $\mathbf{g^x}$, encryptions of the commitments $\mathbf{ctr}$, and proofs π_{ctr}. If the voter has two separate devices for casting and verification, the voting cast device generates the signing key pair while the voting verification device generates the trapdoor key material. Note that verifiability holds if at least one of the voting cast device and the verification device is trusted. The bulletin board $\mathcal{BB}$ is initialised with the voter's id, pk, and π_{sk}.

Voting phase: The voter makes a choice of a candidate $v_0 \in \mathbb{V}$ and generates the ballot β using $\mathsf{Vote}(id, sk, pk_{EA}, (v_0, \mathbf{ctr}, \pi_{\mathbf{ctr}}))$. The voter then submits their ballot to $\mathcal{BB}$, which updates the bulletin board using $\mathsf{Append}(\mathcal{BB}, \beta)$. The voter can verify that their vote is included in the bulletin board by running $\perp/\top \leftarrow \mathsf{VerifyVote}(\mathcal{BB}, (id, pk))$.

Tallying phase: The tallying servers run $\beta_e \leftarrow \mathsf{ExtendMix}(\mathcal{BB}, pk_{EA}, id)$ and then $(R, \Pi) \leftarrow \mathsf{Tally}(\mathcal{BB}, sk_{EA})$ to produce the final tally R. The voter can verify that their vote in the final tally by running $\perp/\top \leftarrow \mathsf{VerifyVote}(\mathcal{BB}, (id, pk), v_0, x_{v_0})$. Anyone can verify the correctness of the final tally result by running the algorithm $\perp/\top \leftarrow \mathsf{VerifyTally}(\mathcal{BB}, (R, \Pi))$.

3.4 Multiple Candidate Elections

In case of an election with 1-out-of-N candidates (especially where N is large), we can immediately apply the method described above by submitting encryptions and trapdoor keys for each candidate, but the efficiency would scale with N.

Additionally, we here describe how to get an efficiency scaling with $l = \lceil \log_2 N \rceil$ but at the cost of transparency in the verification. To this end, we encode the candidate choices in terms of a bit string $b_{i\,i=1,\ldots,l}$ of length l. The idea is that we use the above construction for each b_i. The voter ID_1 submits for all $i = 1, \ldots, N$

$$ID_1, PK_1, \{\mathbf{b_i}\}, \{b_i \oplus 1\}, \{g^{x_{i11}}\}, \{g^{x_{i12}}\}$$

together with an overall zero-knowledge proof that $b_{i\,i=1,\ldots,l}$ is a valid candidate.

From this the voting authorities again copy and re-encrypt the true value b_i together with a proof that is done correctly

$$ID_1, PK_1, \{\mathbf{b_i}\}, \{b_i \oplus 1\}, \{g^{x_{i11}}\}, \{g^{r_{i11}}\}, \{g^{x_{i12}}\}, \{g^{r_{i12}}\}, \{g^{x_{i11}r_{i11}}\}', \{g^{x_{i12}r_{i12}}\}', \{b_i\}', \{a_1\}$$

where a_1 is a random number that is the same for all i. After mixing (independently and separately for each i) and decrypting the a_j terms in the end, we can combine the b_i together to obtain the different candidate choices from the third terms $\{b_i\}'$. The voter can use the first terms to check their candidate bit by bit.

4 Ballot Privacy

We prove that Surtr satisfies ballot privacy. We adapt the game-based security definition of ballot privacy from [Da24]. In Hyperion, the post-tally verification relies on the secret value sent to the voter through a private channel. However, since our scheme does not require a secure channel, we modify the oracle VerifyVote in the definition by eliminating the GetSecret function. Similar to the Hyperion ballot privacy proof, we assume that the bulletin board, tallying server, and the voting devices are trusted, and that each voter is registered with a unique id. This means that these devices do not leak sensitive information, such as the secret signing key, secret trapdoor, or encryption randomness.

Our experiment is shown in Alg.1. The adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ interacts with the voting system through the various oracles. SK and PK are used to store the voter's secret key sk and public key pk. The left and right inputs of the oracle LoR, are stored in V_0 and V_1, respectively, while ST holds the state st associated with the voters' ballots. The adversary $\mathcal{A}_1$ can register honest voters HV using the oracle HonestSetup or register dishonest voters DV with chosen public keys using the oracle DishonestSetup. The adversary $\mathcal{A}_2$ has access to the following oracles:

- LoR, which allows the adversary to have an honest voter cast one of two chosen votes (v_0 or v_1) based on a hidden bit b.
- Tally, which initiates the tallying process but only if the number of votes cast in both scenarios (when $b = 0$ and $b = 1$) is identical.
- VerifyVote, which simulates a voter verifying their vote after the tally.
- Board, which allows the adversary $\mathcal{A}_2$ to obtain a published version of the Bulletin Board $\mathcal{BB}$.
- Cast, which enables $\mathcal{A}_2$ to directly submit a ballot β to $\mathcal{BB}$ for a voter id.
- Verify, which allows $\mathcal{A}_2$ to check the validity of all ballots and tally on $\mathcal{BB}$.

The adversary $\mathcal{A}$ wins the game if it can determine whether the voting system is processing v_0 or v_1 by guessing the hidden value of b. The ballot privacy experiment Alg.1 is designed to model various adversarial capabilities, such as registering dishonest voters, submitting ballots, interacting with the bulletin board, and observing the result of voter verification attempts.

We define the advantage of the adversary $\mathcal{A}$ in the ballot privacy experiment as follows:

$$\mathrm{Adv}_{\mathcal{A}}^{BP} = |Pr[\mathrm{Exp}_{\mathcal{A},\mathcal{VS}}^{BP-0}(\lambda) = 1] - Pr[\mathrm{Exp}_{\mathcal{A},\mathcal{VS}}^{BP-1}(\lambda) = 1]|$$

Theorem 4.1 (Ballot Privacy)**.** *Let $Exp_{\mathcal{A},\mathcal{VS}}^{BP-b}$ Alg.1 be a ballot privacy experiment instantiated with the Surtr algorithms. Then, for any probabilistic polynomial-time adversary $\mathcal{A}$, the advantage against $Exp_{\mathcal{A},\mathcal{VS}}^{BP-b}$ is bounded as follows:*

$$Adv_{\mathcal{A}}^{BP} \leq Adv_{\mathcal{B}}^{\mathsf{IND-1-CCA}} + \epsilon$$

where ϵ is negligible function and $Adv_{\mathcal{B}}^{\mathsf{IND-1-CCA}}$ denotes the advantage of an adversary $\mathcal{B}$ against labelled polynomial-time Indistinguishability Chosen Ciphertext Attack of type 1 (IND-1-CCA) game.

Beweis. In game G_0, we execute the ballot privacy experiment $\mathrm{Exp}_{\mathcal{A},\mathcal{VS}}^{BP-b}$, where the adversary's advantage is denoted by $\mathsf{Adv}_{\mathcal{A}}^{G_0}$.

To initialize Surtr according to the algorithm defined in $\mathrm{Exp}_{\mathcal{A},\mathcal{VS}}^{BP-b}$, the challenger runs the EASetup and Setup algorithms to generate the election's public and secret threshold keys, as well as the trapdoor keys and signing key pairs for the voters. The adversary is given access to the all voters' public signing key PK and the election authority's public encryption key pk_{EA}.

Ballots are generated using the Vote algorithm, which outputs a ballot $\beta = (id, pk, \mathbf{ct}, \mathbf{ctr}, \pi, \sigma)$. The Validate algorithm verifies the voter's identity, the correctness of the ballot's signature, and the validity of the attached proofs for the given ballot β.

The Tally algorithm computes and returns the output of ExtendMix and the final result (R, Π). Finally, the VerifyVote algorithm enables voters to verify their ballots on the bulletin board $\mathcal{BB}$ both before the tallying process and after the final result (R, Π) is published.

In game G_1, the zero-knowledge proofs for the honest voters' ballots are simulated by a challenger, specifically the proof π. Due to the properties of zero-knowledge, the adversary $\mathcal{A}$ has a negligible advantage in distinguishing the simulated proof in game G_1 from the proofs in game G_0. We denote this advantage as ϵ_{zk}. Consequently, the advantage of the adversary $\mathcal{A}$ in distinguishing between games G_0 and G_1 is reduced to the adversary's advantage in the context of the zero-knowledge proofs. Hence we have: $\mathrm{Adv}_{\mathcal{A}}^{G_0} \leq \mathrm{Adv}_{\mathcal{A}}^{G_1} + \epsilon_{zk}$.

The game G_2 is similar to game G_1, with a difference in the oracle Cast where $id \in HV \cup DV$. The adversary $\mathcal{A}$ has a negligible advantage in casting a valid ballot on behalf of an honest voter HV, given that the signature scheme is existentially unforgeable under chosen-message attacks (EUF-CMA). In G_2, the adversary $\mathcal{A}$ can use Cast to cast ballots for voters controlled by $\mathcal{A}$ namely $id \in DV$. In this scenario, the advantage of the adversary $\mathcal{A}$ in distinguishing between games G_1 and G_2 is reduced to the advantage of an adversary in the EUF-CMA security game, denoted $\epsilon_{EUF-CMA}$. Therefore, we have: $\mathrm{Adv}_{\mathcal{A}}^{G_1} \leq \mathrm{Adv}_{\mathcal{A}}^{G_2} + \epsilon_{EUF-CMA}$.

The game G_3 is similar to the game G_2, with the difference that the oracle VerifyVote is assumed to always return $\top$. The correctness of cryptographic protocols ensures that the advantage of the adversary $\mathcal{A}$ in this game remains the same as in G_2, namely $\mathrm{Adv}_{\mathcal{A}}^{G_3} = \mathrm{Adv}_{\mathcal{A}}^{G_2}$.

The game G_4 is similar to game G_3, with one difference in the oracle Tally. In Tally, $ctt_i = \mathsf{ReEnc}(pk_{EA}, ctr_i^t; r_i)$ is computed based on $ctr_i = \mathsf{Enc}(pk_{EA}, g^{x_i}; r_{x_i})$. Indeed, $ctt_i = \mathsf{Enc}(pk_{EA}, g^{x_i t}; r_i + tr_{x_i})$. In G_4 instead of computing ctt_i an uniformly random group element g_i is selected and encrypted, namely, $\mathsf{Enc}(pk_{EA}, g_i; r_{g_i})$. Therefore, the tuple $(c_1, g^{x_i t}, g^t)$ in game G_3 is replaced with (c_1, g_i, g^t). The advantage of the adversary $\mathcal{A}$ in distinguishing game G_4 from game G_3 is reduced to the advantage of an adversary against the DDH problem ϵ_{DDH}. Hence, we have: $\mathrm{Adv}_{\mathcal{A}}^{G_3} \leq \mathrm{Adv}_{\mathcal{A}}^{G_3} + \epsilon_{DDH}$

The game G_5 is similar to G_4, except the real mix-net is replaced with the ideal mix-net functionality. As a result, the adversary's advantage to distinguish between games G_4 and G_5 is now limited to the advantage of differentiating the ideal mix-net from the

real mix-net functionality. We denote this advantage as ϵ_{mix}. Consequently, we have $\text{Adv}_{\mathcal{A}}^{G_4} \leq \text{Adv}_{\mathcal{A}}^{G_5} + \epsilon_{mix}$.

The game G_6 is similiar to the game G_5 with the difference that in the game G_6 we simulate the decryption proofs. Similarly to G_1, the advantage of the adversary $\mathcal{A}$ is reduced to ϵ_{zk}. Therefore we have: $\text{Adv}_{\mathcal{A}}^{G_5} \leq \text{Adv}_{\mathcal{A}}^{G_6} + \epsilon_{zk}$.

In game G_7, we compute the result for honest voters using input from oracle LoR, ensuring that the results of the left and right games are equal. We also perform a decryption of the ballots cast by the adversary $\mathcal{A}$. The final result is determined based on both the adversary's votes and the honest voter's votes. In this game, the advantage of the adversary $\mathcal{A}$ is equivalent to the advantage the adversary $\mathcal{B}$ in the IND-1-CCA security game. This is because the encryption scheme contains non-malleable proofs of knowledge for generating a vote ballot and the signing key that meets IND-1-CCA security [Da24]. Hence, we have: $\text{Adv}_{\mathcal{A}}^{G_6} = \text{Adv}_{\mathcal{A}}^{G_7}$.

Finally, we argue that the advantage of the adversary $\mathcal{A}$ in the game G_0 is reduced to the advantage of the adversary $\mathcal{B}$ in the security game IND-1-CCA.

4.1 Receipt-Freeness

In the construction of Surtr we have focused on the receipt-freeness of the verification mechanism. However, to achieve overall receipt-freeness for Surtr, the vote casting also has to be receipt-free. In our case, the voter is not allowed to know the random coins in the ciphertexts that appear on the bulletin board, containing both the actual vote choice and those encryption of the commitments. One way to achieve this is to use re-randomizable signatures with different labels or public keys for each ciphertext. The ciphertexts are then sent to a server which rerandomizes the ciphertexts and signatures before posting these [Ch16].

5 Verifiability

Here we prove that our proposed scheme, Surtr, satisfies verifiability properties against a malicious bulletin board, talliers, and in the presence of a compromised either secret signing key or secret trapdoor key due to. respectively, a dishonest vote-casting or vote-verifying device. We use the same verifiability definition of Hyperion in [Da24], which also captures post-tally verification of a voter. The post-tally verification allows a voter to verify if

their intended vote was counted in the tally. We assume that the register is trusted in a sense that each voter registered the unique (id, pk). Voters are assumed to have separate devices for vote casting (holding the signing key sk) and vote verification (holding the secret trapdoor verification keys $\mathbf{x}$). The verifiability definition in [Da24] captures the secret leaking information of the voting devices by modeling the honest and corrupted casting and verification devices. The corrupted casting device $\mathcal{D}_c$ leaks the voter's secret signing key sk and the encryptions of the commitments **ctr** and corresponding proofs. The corrupted verification device $\mathcal{D}_v$ leaks the secret trapdoor keys $\mathbf{x}$. We denote by $\mathcal{V}_{chkd}$ the set of voters who verify that a valid ballot with their id being present on $\mathcal{BB}$ and checking their votes after tallying using VerifyVote. Voters whose casting and verification devices are both corrupted are considered as corrupted voters. The definition highlights the adversary's advantage in outputting a valid bulletin board and tally while allowing the adversary to engage in at least one of the following malicious behaviors: i) changing the vote of a voter that belongs to $\mathcal{V}_{chkd} \cap \mathcal{H}_v$ or $\mathcal{V}_{chkd} \cap \mathcal{H}_c$; ii) for voters using honest casting devices, manipulating the process by either stuffing votes or altering a cast vote in a manner other than simply deleting it, specifically for those in $\mathcal{V} \setminus (\mathcal{V}_{chkd} \cap \mathcal{H}_c)$; iii) allowing the remaining eligible voters, namely those in $\mathcal{V}_{chkd} \cap \mathcal{D}_v$ and $\mathcal{D}_c \setminus (\mathcal{V}_{chkd} \cap \mathcal{H}_v)$, to cast more than one vote per voter.

Theorem 5.1 (Verifiability)**.** *Surtr achieves verifiability, assuming the signature scheme is existentially unforgeable under chosen message attacks (EUF-CMA) and the 1-DHI problem is computationally hard.*

Beweis. Assume that the adversary $\mathcal{A}$ outputs $(\mathcal{BB}, R, \Pi)$ such that the verification algorithm $\mathsf{Verify}(\mathcal{BB}, R, \Pi)$ returns $\top$. The correctness of the underlying cryptographic primitives, along with the soundness of the zero-knowledge proofs for the voter's key, ballot well-formness, mix-net, and decryption, ensures that the result R has been correctly computed from the valid ballots in $\mathcal{BB}$.

We aim to prove that the adversary A has a negligible advantage in altering the ballots and the voter's intended vote of a voter with $id \in \mathcal{V}_{chkd} \cap (\mathcal{H}_v \cup \mathcal{H}_c)$. Note that if the verification device $\mathcal{D}_v$ is corrupted, the adversary can use its knowledge of the trapdoor keys $\mathbf{x}$ to generate a matching dual key g^r, causing VerifyVote to return $\top$ even if the ballot has been altered. However, we prove that as long as at least one of the devices is honest, the adversary has a negligible advantage in changing the intended vote without being detected.

We first prove that the adversary $\mathcal{A}$ has a negligible advantage in altering the ballots of the voter $id \in \mathcal{V}_{chkd} \cap \mathcal{H}_c$. furthermore, the ballots of the voter $id \in \mathcal{H}_c$ can only be deleted.

If $id \in \mathcal{H}_c$, the adversary $\mathcal{A}$ does not know the voter's secret signing key sk. Therefore, the advantage of $\mathcal{A}$ in generating valid ballots for voters with $id \in \mathcal{H}_c$ is reduced to EUF-CMA security of the signature scheme. This implies that for voters with an honest vote-casting device who did not check their vote, the adversary $\mathcal{A}$ has a negligible advantage in ballot stuffing and can only delete a ballot. Hence, for all voters in $\mathcal{H}_c$, the ballots cannot be

altered, but they can be removed. Consequently, for all successfully checked voters with honest vote-casting devices, i.e., $id \in \mathcal{V}_{\text{Chkd}} \cap \mathcal{H}_c$, all corresponding ballots must appear exactly as they were cast.

Now we prove that the advantage of the adversary $\mathcal{A}$ in changing the intended vote of the voter with $id \in \mathcal{V}_{chkd} \cap \mathcal{H}_v$ can be reduced to to the advantage of the adversary $\mathcal{B}$ in the computational 1-DHI problem, where $\text{Adv}_{\mathcal{B}}^{\text{1-DHI}} = \Pr\left[x \leftarrow \mathbb{Z}_q : g^{1/x} = \mathcal{A}(g^x)\right]$.

Let $id \in \mathcal{V}_{chkd} \cap \mathcal{H}_v$. Since the verification device $\mathcal{H}_v$ is honest, the adversary $\mathcal{A}$ does not have the trapdoor keys $\mathbf{x}$ associated with the voter id. However, the adversary may have access to the voter's secret signing key sk and $(g^{x_0}, ..., g^{x_{N-1}})$ by decrypting $\mathbf{ctr}$.

Let $\beta = (id, pk, \mathbf{ct}, \mathbf{ctr}, \pi, \sigma)$ be a ballot of voter id in our scheme, with the intended vote v_0 with corresponding trapdoor key x_0. Assume that the adversary generates a valid ballot such that the voter's intended vote is altered. For example, $\mathcal{A}$ may change the order of the vote ciphertexts $\mathbf{ct}$, resulting in a different intended vote. We show that $\mathcal{A}$ has a negligible advantage in generating a dual key $dk = g^r$ without the knowlege of x_0 such that the post-tally verification $\mathsf{VerifyVote}(\mathcal{BB}, (id, pk), v_0, x_0) \rightarrow \top$. This implies that $\mathcal{A}$ has changed the voter's intended vote while successfully evading detection during verification.

Assume that the adversary $\mathcal{A}$ generates a valid β' where the vote is changed to v_1 with the corresponding trapdoor key x_0. Assume that $\mathcal{A}$ generates a dual key $dk = g^r$ for this voter such that dk^{x_0} verifies the vote v_0. This implies that there exists a commitment $h_i^r = g^{x_i r}$ on $\mathcal{BB}$ where $x_i \neq x_0$, such that $dk^{x_0} = g^{x_i r}$. If $x_i r$ is known, one can compute $g^{(1/x_0)} = dk^{(1/(x_i r))}$. This means that the adversary $\mathcal{B}$ who is given g^{x_0} as an instance of the computational 1-DHI problem, can leverage the advantage of adversary $\mathcal{A}$ to solve the computational 1-DHI problem. The adversary $\mathcal{B}$ simulates the verifiability experiment for the adversary $\mathcal{A}$. To do this, $\mathcal{B}$ can extract the secret trapdoor key for all voters with corrupted casting devices, using the simulation-sound extractability property of the zero-knowledge proof of knowledge.

Finally there are ballots on $\mathcal{BB}$ that the adversary $\mathcal{A}$ can select the votes. For a voter with $id \in \mathcal{D}_v$ such that $id \notin \mathcal{V}_{chkd}$, the adversary $\mathcal{A}$ can either change the vote or delete the ballot. However, if $id \in \mathcal{D}_v$ and $id \in \mathcal{V}_{chkd} \cap \mathcal{D}_v$, then there exists a valid ballot on $\mathcal{BB}$ in which the intended vote has been altered.

let $\mathcal{M} = \mathcal{V}_{chkd} \cap (\mathcal{H}_v \cup \mathcal{H}_c)$, $\mathcal{S} \subseteq \mathcal{V} \setminus (\mathcal{V}_{chkd} \cap \mathcal{H}_c)$ and $n \in [|\mathcal{V}_{chkd} \cap \mathcal{D}_v|, \ldots, |\mathcal{D}_c \setminus (\mathcal{V}_{chkd} \cap \mathcal{H}_v)|]$ which denotes the number of votes defined by $\mathcal{A}$. Therefore we have:

$$R = \rho\left([V[j]]_{j \in \mathcal{S}}\right) \star \rho\left([v_j]_{j=1}^{n}\right) \star \rho\left([V[j]]_{j \in \mathcal{M}}\right)$$

where $V[j]$ denotes the intended vote of the voter j and $[v_j]_{j=1}^{n}$ are the votes chosen by the adversary $\mathcal{A}$.

Tab. 1: Performance of Surtr.

nr. of voters	1000	10000	100000	1000000
Setup	0.01s	0.01s	0.01s	0.01s
Voting (avg.)	0.02s	0.02s	0.02s	0.02s
Ballot extension and re-enc. mix	29s	5m	50m	8h 24m
Decr. mix	2.95s	31s	12m	13h
Total tallying	32s	5m 31s	1h 12m	21h 24m

6 Performance

We implement a prototype of Surtr in Python [GSR], using the Hyperion implementation [Da24] as a starting point. We run our experiments on a 2020 Intel Xeon with 96 cores running at 3GHz. Table 1 shows the runtime results for Setup, Voting, and Tallying phases, considering two candidates and three tellers, with a threshold of two. For the tallying phase, we distinguish between the time required to add encryptions and proofs to each voter's ballot and to perform the re-encryption mixes, versus the time needed to execute the decryption mixnet.

Setup and Voting phases are clearly unaffected by the number of voters. Tallying complexity of Surtr is linear in both the number of voters and the number of candidates[4]. In contrast, tallying in Hyperion without individual views is linear in the number of voters. However, Hyperion requires individual views to prevent collision of commitments. In this case, Surtr outperforms Hyperion, as it avoids collisions by design and does not require individual views, which rely on exponentiation mixes that scale with the number of votes.

7 Conclusion

We presented Surtr, a new voting scheme that achieves transparent verification and coercion mitigation without relying on a private notification channel. Surtr is based on standard cryptographic primitives and elegantly avoids the tracker collision problem that undermined the Selene scheme without requiring exponentiation mixes as in Hyperion.

Voters under coercion may feel more comfortable using Surtr, as they are not required to generate a fake dual key, unlike in Hyperion. We have proven that Surtr satisfies ballot privacy and verifiability, and we have outlined how it can be made fully receipt-free. Our prototype implementation demonstrates that it can scale to millions of voters. Multiple candiate elections can also get benefit from a logarithmic speedup, if needed. Future work includes developing a post-quantum secure version of Surtr and conducting user studies to assess the usability of the scheme.

[4] The slowdown for 1 million voters in the decryption mix is due to RAM limits.

Literaturverzeichnis

[ACW13] Arnaud, M.; Cortier, V.; Wiedling, C.: Analysis of an electronic boardroom voting system. In: International Conference on E-Voting and Identity. Springer, 2013.

[AFT08] Araújo, R.; Foulle, S.; Traoré, J.: A practical and secure coercion-resistant scheme for remote elections. In: Dagstuhl Seminar Proceedings. 2008.

[AS20] Alsadi, M.; Schneider, S.: Verify my vote: voter experience. E-Vote-ID, S. 280, 2020.

[CGY22] Cortier, V.; Gaudry, P.; Yang, Q.: Is the JCJ voting system really coercion-resistant? Cryptology ePrint Archive, 2022.

[CH12] Clark, J.; Hengartner, U.: Selections: Internet Voting with Over-the-Shoulder Coercion-Resistance. In (Danezis, G., Hrsg.): Financial Cryptography and Data Security. Springer Berlin Heidelberg, Berlin, Heidelberg, S. 47–61, 2012.

[Ch16] Chaidos, P. et al.: BeleniosRF: A non-interactive receipt-free electronic voting scheme. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. S. 1614–1625, 2016.

[Da24] Damodaran, A. et al.: Hyperion: transparent end-to-end verifiable voting with coercion mitigation. Cryptology ePrint Archive, 2024.

[Di19] Distler, V. et al.: Security-Visible, Yet Unseen? In: Proceedings of the 2019 CHI conference on human factors in computing systems. S. 1–13, 2019.

[GGS24] Giustolisi, R.; Garjan, M. S.; Schuermann, C.: Thwarting Last-Minute Voter Coercion. In: 2024 IEEE Symposium on Security and Privacy (SP). 2024.

[GSR] Giustolisi, R.; Sheikhi Garjan, M.; Roenne Browne, P.: Surtr prototype implementation, https://github.com/fgiustol/Surtr, 2025-05-18.

[JCJ10] Juels, A.; Catalano, D.; Jakobsson, M.: Coercion-resistant electronic elections. In: Towards Trustworthy Elections. Springer, S. 37–63, 2010.

[Kü16] Küsters, R. et al.: sElect: a lightweight verifiable remote voting system. In: CSF. IEEE, S. 341–354, 2016.

[LHK16] Locher, P.; Haenni, R.; Koenig, R. E.: Coercion-resistant internet voting with everlasting privacy. In: Financial Cryptography and Data Security: FC 2016 International Workshops. Springer, S. 161–175, 2016.

[Mo24] Mosaheb, R. et al.: Direct and Transparent Voter Verification with Everlasting Receipt-Freeness. In: E-VOTE-ID. Springer, 2024.

[RRI16] Ryan, P. Y. A.; Rønne, P. B.; Iovino, V.: Selene: Voting with transparent verifiability and coercion-mitigation. In: International Conference on Financial Cryptography and Data Security. Springer, S. 176–192, 2016.

[Sa20] Sallal, M. et al.: Augmenting an internet voting system with selene verifiability using permissioned distributed ledger. In: 2020 IEEE 40th ICDCS. 2020.

[Sp12] Spycher, O. et al.: A new approach towards coercion-resistant remote e-voting in linear time. In: Financial Cryptography and Data Security. Springer, S. 182–189, 2012.

[TW10] Terelius, B.; Wikström, D.: Proofs of Restricted Shuffles. In: Progress in Cryptology – AFRICACRYPT 2010. Springer, S. 100–113, 2010.

[Zo21] Zollinger, M.-L. et al.: User Experience Design for E-Voting: How mental models align with security mechanisms. arXiv preprint arXiv:2105.14901, 2021.

Algorithm 1 $ExpBP^{b}_{\mathcal{A},\mathcal{VS}}$

$SK, PK, ST, V0, V1 \leftarrow []_{\perp}$
$HV, DV \leftarrow \{\}$
$(pk_{EA}, sk_{EA}) \leftarrow \mathsf{EASetup}()$
$st_{\mathcal{A}} \leftarrow \mathcal{A}_1(pk_{EA}, PK)$
$b' \leftarrow \mathcal{A}_2(st_{\mathcal{A}})$
Return $b' = b$

Oracle HonestSetup (id) :
$(sk, pk) \leftarrow \mathsf{Setup}(id, pk_{EA})$
If $\mathsf{ValidCred}(id, pk, pk_{EA}) = \perp$ **Return** $\perp$
Else $SK[id] \leftarrow sk, PK[id] \leftarrow pk$ **and** $HV \xleftarrow{\cup} \{id\}$

Oracle DishonestSetup(id) :
If $id \in HV$ or $\mathsf{ValidCred}(id, pk, pk_{EA}) = \perp$ **Return** $\perp$
Else $PK[id] \leftarrow pk$ **and** $DV \xleftarrow{\cup} \{id\}$ **End If**

Oracle LoR (id, v_0, v_1) :
If $id \notin HV$ **Return** $\perp$
Else $sk \leftarrow SK[id];\ pk \leftarrow PK[id]$
$(\beta, st) \leftarrow \mathsf{Vote}(id, sk, pk_{EA}, v_b)$ **End If**
If $\mathsf{Validate}(\mathcal{BB}, \beta) = \perp$ **Return** $\perp$
Else$V_0[id] \leftarrow v_0;\ V_1[id] \leftarrow v_1$
$ST[id] \leftarrow st;\ \mathcal{BB}[id] \leftarrow \beta$ **End If**

Oracle Tally():
If $\rho(V_0) = \rho(V_1)$ **Return** $\mathsf{Tally}((\mathcal{BB}, pk_{EA}, PK), sk_{EA})$ **End If**

Oracle VerifyVote(id) :
If $id \notin HV \cup DV$ **Return** $\perp$
Else $pk \leftarrow PK[id]$
$(R, \Pi) \leftarrow \mathsf{Tally}()$
$sk \leftarrow SK[id]$ **and** $st \leftarrow ST[id]$
Return $\mathsf{VerifyVote}(\mathcal{BB}, (id, pk, st), v, s)$ **End If**

Oracle Board():
$\mathcal{BB}' \leftarrow \mathsf{Publish}(\mathcal{BB})$
Return $\mathcal{BB}'$

Oracle Cast (id, β) :
If $id \notin HV \cup DV$ **or** $\mathsf{Validate}(\mathcal{BB}, \beta) = \perp$ **Return** $\perp$
Else $\mathcal{BB}[id] \leftarrow \beta$ **End If**

Oracle Verify() :
Return $\mathsf{Verify}(\mathcal{BB}, (R, \Pi))$

Algorithm 1: Ballot privacy experiment. The adversary $\mathcal{A}$ wins if $b' = b$.